Lezioni Algoritmi e strutture Dati

Alberto Bernasconi

a.a 25/26

Capitolo 1

Introduzione

introduzione al Corso di Algoritmi e strutture Dati.

23 Febbraio 2026 Introduzione, analisi di due esempi di problemi da risolvere

1.1 Il problema della celebrita'

Struttura di un problema:

- Partiamo da una **definizione** formale: *Una celebrità e una persona che non conosce nessuo ma è conosciuta da tutti*, astrazione di simmetria estrema tra le persone conosciute dalla celebrità e quelle che ella stessa conosce. Assunzione, che un a celebrita non conosce nessuno
- **Problema**[1]: *Dato un insieme di n persone, determinare se questo contiene una celebrità (tramite domande del tipo "X conosce Y")*; Pensiamo ad una campagna di marketing, la scelta di un testimonial in una pubblicità e basata sulla popolarità dell'individuo, si incarica un'agenzia di trovare questa persona.
- **Soluzione Naive**[2]: *Si esaminano le persone una ad una e per ceiascuna di questa si verifica se è una celebrita (tramite 2(n-1) domane)* ipotizziamo che qual'ora vi sia una celebrità sia unica, ma quante domande devo fare per comprendere se una persona è la nostra eletta.

Il numero di domande che devo fare sara:

$$2 \cdot n(n-1)$$

Se supponiamo che n sia un numero abbastanza grande, ma nemmeno troppo, se colui che pagato per trovare questa persona anche prendendo una minima cifra per ogni intervista, viene a raggiungere dato il numero di persone un costo elevato rispetto al natura del problema.
Come possiamo fare meglio ?

- Si seleziona una coppia di persone a caso (X,Y) e si pone la domanda "X conosce Y?" se la risposta è affermativa X non è una celebritàè altrimenti Y non è una celebrità.

[1] **Problema:** Situazione da risolvere con maggiore efficienza e rapidità possibile
[2] **Soluzione Naive:** La soluzione più semplice e penabile ma quasi mai la più efficiente

- Si ripete questo passo precedente su un insieme di $n-1$ persone (con una domanda si elimina una persona)
- Dopo $n-1$ passi si verifica l'unica persona rimasta sia una celebrita (con $2n-2$ domande)

Gia in questo modo riusciamo a ridurre significativamente il costo della ricerca rispetto alla **Soluzione naive**.

La soluzione: viene trovata tramite la soluzione di una particolare **equazione di ricorrenza**[3]

$$c_n = c_{n-1} + 1, \qquad c_1 = n-1 \quad \Rightarrow \quad c_n = 2n-2$$

Quale era il nostro **Obbietivo** iniziale, raggiungere l'**efficienza** della soluzione.

1.1.1 Il problema del missionario chirurgo

Struttura di un problema:

- **Partiamo da una definizione formale:** Un missionario chirurgo opera in missioni con scarse risorse e deve ottimizzarne l'uso. Per garantire la sicurezza ed evitare infezioni, il chirurgo non deve mai venire a contatto diretto con i tessuti del paziente, utilizzando guanti in lattice sterili.
- **Problema:** Avendo a disposizione un numero limitato di risorse, ovvero n paia di guanti, quanti interventi sicuri è possibile eseguire? A differenza del problema della celebrità (dove si volevano minimizzare le risorse), qui l'approccio è duale: le risorse sono fissate in partenza e l'obiettivo è massimizzare il numero di interventi.
- **Soluzione Naive**[4]**:** Si esegue un intervento consumando un singolo paio di guanti alla volta, per un totale di n interventi. Questo approccio genera uno spreco, poiché per ogni intervento la mano del chirurgo contamina una nuova superficie interna dei guanti.
- **Limite teorico:** Fissate n paia di guanti, abbiamo a disposizione un totale di $2n$ superfici (contando sia l'interno che l'esterno di ciascun guanto). Poiché almeno una superficie deve necessariamente entrare in contatto con la mano del chirurgo (diventando infetta), rimangono al massimo $2n-1$ superfici sterili. Di conseguenza, il numero massimo teorico di interventi possibili è $2n-1$.
- **Miglioramento (Colpo di genio):** Per superare la soluzione naive, si sfrutta la possibilità di infilare i guanti uno sopra l'altro.
  - *Fase di andata*: Si infilano tutte le n paia di guanti una sull'altra ed si esegue il primo intervento. Finito l'intervento, si sfila il guanto più esterno, lo si rovescia (mantenendo all'esterno una superficie sterile) e lo si mette da parte. Si eseguono così interventi a scalare fino a rimanere con un solo paio di guanti infilato, completando n interventi in questa fase.
  - *Fase di ritorno*: Arrivati a 1, si ricomincia a infilare uno alla volta i guanti precedentemente sfilati e rovesciati. La superficie interna (contaminata) del guanto recuperato andrà a contatto con la superficie esterna (anch'essa contaminata) del guanto già sulle mani, lasciando esposta la superficie esterna sterile. Questo permette di eseguire altri $n-1$ interventi.

[3] **equazione di ricorrenza:** equazione dove un valore dipende dalla soluzione dello stesso problema ma in una dimensione più piccola

[4] Soluzione Naive: La soluzione più immediata ma meno efficiente.

- **La soluzione:** Viene espressa tramite una particolare equazione di ricorrenza che descrive gli interventi effettuati nella fase di andata e ritorno:

$$C_n = C_{n-1} + 2, \quad C_1 = 1 \implies C_n = 2n - 1$$

  Con questo schema ottimalizziamo le risorse raggiungendo esattamente il limite teorico di $2n - 1$ interventi, dimostrando che nessun'altra soluzione può fare di meglio.

1.1.2 Fair division 53.00, il problema del comunista

Struttura di un problema:

- **Partiamo da una definizione formale:** L'equadivisione (Fair division) consiste nel trovare uno schema per spartire un insieme di risorse (oggetti o beni) con un certo valore complessivo R tra K individui (o entità) in maniera equa.
- **Vincoli del problema:** Un algoritmo per la ripartizione deve soddisfare rigorosamente tre regole fondamentali:
  1. **Nessuna coalizione:** Deve essere matematicamente impossibile che un gruppo si metta d'accordo per danneggiare un altro individuo.
  2. **Responsabilità personale:** Se un individuo riceve meno della sua quota spettante (ovvero una frazione minore di R/K), la responsabilità deve essere esclusivamente sua, frutto di un suo errore di scelta o divisione.
  3. **Assenza di invidia (*Envy-free*):** Se a un individuo viene assegnato un valore che ritiene almeno pari al dovuto, egli non ha diritto di invidiare ciò che è stato assegnato agli altri.
- **Soluzione per $K = 2$:** Si adotta il classico metodo "uno divide, l'altro sceglie". L'individuo A divide le risorse in due parti, cercando di farle equivalenti. L'individuo B sceglie per primo. Se A sbaglia a dividere e subisce un danno (rimanendo con la parte minore), la colpa è sua; se B prende la parte maggiore non c'è invidia, e non ci sono coalizioni.
- **Soluzione per $K = 3$:** Il processo richiede un algoritmo leggermente più articolato:
  - L'individuo A divide le risorse in tre insiemi disgiunti S_1, S_2, S_3, cercando di fare in modo che ciascuno valga $R/3$.
  - L'individuo B esamina le parti e indica l'insieme S_i che reputa di minor valore (quindi con un valore stimato $\leq R/3$).
  - L'individuo C esamina a sua volta l'insieme S_i indicato da B:
    * Se C concorda che $S_i \leq R/3$, l'insieme S_i viene assegnato ad A. Ciò che rimane viene riunito e diviso tra B e C applicando lo schema per $K = 2$.
    * Se C ritiene invece che $S_i > R/3$, allora l'insieme S_i viene assegnato a C (che riceve più di un terzo e non può lamentarsi). Ciò che rimane viene spartito tra A e B.
- **Varianti e compensazioni:**
  - *Beni continui (Esempio della birra in pizzeria):* Per dividere una quantità continua (come 5 litri di birra) tra K persone, il primo versa in un boccale la dose che ritiene corretta. Il boccale gira: se chi lo riceve ritiene la dose eccessiva, ne rovescia un po' e lo passa al compagno successivo. Il boccale va all'ultima persona che ha ridotto la quantità (o al primo se nessuno è intervenuto). Chi lo ottiene è soddisfatto, e gli altri non possono lamentarsi perché hanno avuto il loro turno per intervenire.

- *Beni discreti indivisibili:* Qualora gli oggetti non possano essere divisi in parti di egual valore (es. un tablet e un piccolo gadget), si fa ricorso a tecniche di compensazione integrando il valore da spartire con del denaro.

- **Conclusione e scopo introduttivo:** I problemi della celebrità, del missionario chirurgo e della Fair division servono a illustrare il filo conduttore dell'informatica algoritmica: **partire sempre da una definizione formale, progettare uno schema corretto (algoritmo) per risolverlo e prestare un'attenzione ossessiva all'efficienza e all'uso delle risorse**. Un metodo teoricamente corretto, ma che richiede risorse inaccettabili, è nella pratica inutile tanto quanto un metodo sbagliato.

24 Febbraio 2026 Introduzione alla Complessità Algoritmica

1.2.1 Problemi, Funzioni e Algoritmi

Struttura e definizioni:

- **Problema formale:** Un problema associa un insieme di istanze di ingresso a un insieme di soluzioni, rappresentabile matematicamente come una funzione $f(x)$. Ad esempio, nel problema della primalità, l'input è un numero naturale e l'output è un valore booleano (1 se primo, 0 se composto).

- **Algoritmo vs Programma:** L'algoritmo è il metodo astratto per risolvere il problema (associabile a una funzione parziale che prevede il simbolo di indefinito $\perp$ per i cicli infiniti), mentre il programma è la sua implementazione pratica in un dato linguaggio per uno specifico esecutore hardware.

- **Correttezza:** Un algoritmo è considerato corretto se il suo dominio di input coincide con le istanze del problema, il suo output coincide con le soluzioni, e se la funzione calcolata dall'algoritmo è esattamente uguale a quella del problema per ogni input. Nell'analisi algoritmica di base ci si concentra esclusivamente su algoritmi corretti.

1.2.2 Modello di calcolo e Dimensione dell'Input

Valutazione delle risorse:

- **Indipendenza dall'hardware:** Per misurare l'efficienza non si utilizza il tempo cronologico (secondi), poiché questo dipenderebbe dalla velocità del processore. La complessità viene valutata contando il numero di istruzioni elementari eseguite (complessità di tempo) e le celle di memoria occupate (complessità di spazio).

- **Dimensione del dato (n):** La complessità dipende strettamente dalla grandezza dell'input, definita attraverso una "funzione peso" (W). Questa può rappresentare il numero di cifre per un intero, la lunghezza di un vettore di dati, o l'ordine di una matrice.

1.2.3 Le tre Analisi di Complessità

Gestire la variabilità delle istanze: A parità di dimensione n, non tutte le istanze richiedono lo stesso numero di istruzioni. Si definiscono quindi tre tipi di analisi:

- **Caso Peggiore (*Worst-case*):** Valuta il costo massimo tra tutte le istanze di dimensione n (associato all'utente "sfortunato", metaforicamente Paperino). È fondamentale per i sistemi critici (es. controllo di centrali nucleari), dove le tempistiche di sicurezza devono essere garantite in ogni situazione.

- **Caso Migliore (*Best-case*):** Valuta il costo minimo tra tutte le istanze di dimensione n (associato alla "fortuna", es. Gastone) [18, 22]. Un esempio è l'ordinamento di un vettore che risulta essere già ordinato in partenza [23, 24].

- **Caso Medio (*Average-case*):** Esprime il consumo atteso calcolando una media pesata basata sulle probabilità di ogni singola istanza [25, 26]. È il metro di giudizio principale per i sistemi di transazioni (es. sportelli bancomat), che devono garantire buone prestazioni per la maggior parte del tempo [27-29].

  - *Raggruppamento per costo:* La formula del caso medio può essere semplificata raccogliendo a fattor comune le istanze che hanno lo stesso costo K [30-32].
  - *Ipotesi di equiprobabilità:* Se non si conosce la distribuzione reale dei dati, si assume che tutte le istanze siano equiprobabili. In questo caso, la probabilità diventa il rapporto $\frac{\sigma_{n,K}}{|I_n|}$, dove $\sigma_{n,K}$ è il numero di istanze di dimensione n con costo K, e $|I_n|$ è il numero totale di istanze [33-36].

1.2.4 Impatto della Tecnologia (Hardware vs Software)

L'importanza di un buon algoritmo: Migliorare l'hardware non elimina la necessità di algoritmi efficienti. Ipotizzando l'arrivo di una macchina 10 volte più veloce [37-39]:

- Con un **algoritmo lineare ($O(n)$)**, la dimensione massima del problema risolvibile cresce dello stesso fattore (10 volte più grande) [39].

- Con un **algoritmo quadratico ($O(n^2)$)**, la dimensione massima cresce proporzionalmente alla $\sqrt{10}$ (circa 3,16 volte) [40].

- Con un **algoritmo esponenziale ($O(2^n)$)**, una macchina 10 volte più veloce permette di aumentare la dimensione del problema solo dell'addendo $\log_2(10) \approx 3,32$ [40, 41].

Ciò dimostra che algoritmi inefficaci (esponenziali) vanificano i progressi tecnologici dell'hardware. Per sfruttare la potenza dei nuovi computer, è essenziale progettare algoritmi con complessità polinomiale [42].

26 Febbraio 2026 Le Notazioni Asintotiche e il Modello RAM

1.3.1 Introduzione alle Notazioni Asintotiche

Scopo e definizioni:

- **L'obiettivo:** Calcolare in modo esatto il numero di istruzioni eseguite da un algoritmo è un compito arduo, perciò ci si accontenta di comprenderne l'andamento asintotico, ovvero come crescono le risorse richieste al crescere della dimensione dell'input n. Questo permette di semplificare i calcoli sacrificando l'estrema precisione a favore dell'ordine di grandezza.

- **O grande (O - Limite superiore):** Una funzione $f(n)$ è $O(g(n))$ se esistono un intero $n_0 \geq 0$ e una costante positiva $c > 0$ tali che $f(n) \leq c \cdot g(n)$ per ogni $n \geq n_0$. Intuitivamente, significa che la funzione f è dominata superiormente da g da un certo punto in poi. Questa notazione è ideale per descrivere la complessità nel **caso peggiore**, poiché indica il limite massimo di crescita. Le costanti moltiplicative non vengono catturate dalla notazione, quindi scrivere $O(n)$ o $O(10n)$ è equivalente.

- **Omega grande (Ω - Limite inferiore):** È la notazione simmetrica a O grande. Si dice che $f(n)$ è $\Omega(g(n))$ se esistono n_0 e una costante $c > 0$ tali che $f(n) \geq c \cdot g(n)$ per ogni $n \geq n_0$. In questo caso, è la funzione f a dominare g e a stare sopra il suo grafico.

- **Theta grande (Θ - Stesso ordine di grandezza):** Una funzione $f(n)$ è $\Theta(g(n))$ se è contemporaneamente $O(g(n))$ e $\Omega(g(n))$. In formule, significa che $f(n)$ è compresa tra $d \cdot g(n)$ e $c \cdot g(n)$ per due costanti $c, d > 0$ da un certo n_0 in poi. Indica che due funzioni crescono con lo stesso tasso, trascurando i coefficienti moltiplicativi e i termini di grado inferiore.

- **Asintotico a ($\sim$ - Uguaglianza asintotica):** Si scrive $f(n) \sim g(n)$ quando il limite del rapporto $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ è uguale a 1. È una notazione molto più precisa di Θ, poiché garantisce che le due funzioni abbiano non solo lo stesso ordine di grandezza, ma anche lo stesso identico coefficiente moltiplicativo.

- **O piccolo (o - Funzione trascurabile):** Si dice che $f(n)$ è $o(g(n))$ se il limite del rapporto $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ è uguale a 0. Questo denota che il numeratore $f(n)$ cresce in modo così lento rispetto a $g(n)$ da diventare trascurabile al crescere di n.

1.3.2 Proprietà e Regole di Calcolo

Semplificazione dell'analisi algoritmica:

- **Relazioni di equivalenza:** Le notazioni Θ e $\sim$ soddisfano le proprietà riflessiva, simmetrica e transitiva, generando relazioni di equivalenza che partizionano l'infinito mondo delle funzioni in classi di equivalenza ben definite.

- **Costanti e Somma:** Se $f(n) = O(g(n))$, allora anche $c \cdot f(n) = O(g(n))$ per qualunque costante c. La somma di due funzioni $O(g_1) + O(g_2)$ produce una complessità pari a $O(g_1 + g_2)$, che può essere ulteriormente semplificata mantenendo unicamente il termine con la crescita asintotica maggiore. Questa proprietà è fondamentale: analizzando un software con molteplici procedure, ci si può concentrare esclusivamente sul frammento di codice più costoso, buttando via i termini inferiori.

- **Prodotto e Differenza:** Il prodotto $O(g_1) \cdot O(g_2)$ risulta in $O(g_1 \cdot g_2)$. Al contrario, tali regole **non** si applicano alla differenza: in generale, la differenza tra due O non coincide con l'O della loro differenza, un fatto facilmente dimostrabile tramite controesempi.

1.3.3 Il Modello di Calcolo RAM (Random Access Machine)

Caratteristiche dell'esecutore astratto:

- **Definizione e scopo:** Per studiare formalmente gli algoritmi e calcolarne la complessità in modo univoco, serve un modello di esecutore astratto semplice e pulito, ovvero la Macchina RAM (Random Access Machine). Il termine "Random Access" indica che il tempo di accesso per qualsiasi informazione in memoria è costante e non varia in base alla posizione fisica dell'indirizzo.

- **Struttura Hardware semplificata:**
  - **Memoria Dati (RAM):** Consiste in una sequenza infinita di registri numerati a partire da zero ($R_0, R_1, R_2, \dots$). L'assunzione che i registri siano infiniti ed in grado di contenere interi arbitrariamente grandi serve per slegare il modello dai vincoli tecnologici e fisici dell'hardware reale.

- **Memoria Programma (ROM):** Le istruzioni del programma sono caricate in una memoria separata di sola lettura, numerate in progressione.
- **Program Counter (LC):** È il registro "Location Counter", che conserva l'etichetta (l'indirizzo intero) dell'istruzione corrente che la macchina deve eseguire.
- **I/O (Nastri):** La comunicazione della macchina avviene mediante due nastri: un nastro di input (sola lettura, dove si precaricano i dati) e un nastro di output (sola scrittura per i risultati). Entrambi i nastri vengono percorsi da testine che si muovono esclusivamente in un'unica direzione, da sinistra verso destra. Ogni cella di questi nastri ospita un singolo numero intero.
- **Accumulatore e ALU:** Tra le memorie, il registro zero (R_0) assume lo speciale ruolo di Accumulatore, mentre la ALU funge da circuito dedicato a svolgere e risolvere le classiche operazioni aritmetiche e logiche previste dal set di istruzioni.

- **Set di istruzioni e Costo:** Il modello possiede pochissime istruzioni basilari: I/O, operazioni aritmetiche, accesso in memoria e salti (jump). Il costo spaziale è determinato dai registri allocati, mentre quello temporale dal numero di istruzioni calcolate. Tuttavia, dovendo la macchina poter ospitare numeri indefinitamente immensi, a volte non basta pesare ogni operazione come "1" e diventa necessario ricorrere al "criterio di costo logaritmico", basato sull'effettiva lunghezza dell'intero da elaborare.

02 Marzo 2026 — Modello Ram

Un programma è una struttura finita di istruzioni, la sua etichetta sara contenuto nel registro e che prima o poi dobra essere eseguita.

- é una sequenza finita di istruzioni
- ogni istruzione ha un etichetta (l'indirizzo contenuto in lc)
- ogni istruzione è una coppia (opcode, indirizzo)
- ogni indirizzo può essere un operando o un'etichetta

Vedi slide tipologia,Opcode, Indirizzo per esempi tabella da copiare

1.1.3 Etichette

- sono associate solo ai comandi di salto;
- indicano a quale istruzione viene pasato il controllo.

Esempio: **minuto 14**

```
1  READ  0
2  ADD  =1
     :
     :
K  JUMP  2
```

Un operando può assumere tre forme diverse: **Vedi slide con appunti minuto 18**

1.1.4 Stato della macchina